CORTEX

Industrial Knowledge Intelligence Platform

Technical Architecture and Implementation Report

Project	Cortex
Version	1.0 — ET Hackathon Build
Date	July 07, 2026
Stack	FastAPI · Next.js · Neo4j · Qdrant · Ollama · PostgreSQL
Modules	Ingestion · Copilot · RCA · Compliance · Lessons Learned

EXECUTIVE SUMMARY

What Cortex Is and What It Does

Cortex is a full-stack industrial knowledge management platform built for asset-intensive operations environments. The core problem it solves is straightforward to state but hard to fix: critical operational knowledge is scattered across PDF manuals, Excel inspection sheets, email threads, incident reports, and OEM documents, and the people who need that information spend a disproportionate share of their working hours hunting for it rather than using it.

A 2024 McKinsey survey found that professionals in these industries spend roughly 35 percent of their time on information retrieval or recreation of documents that already exist. A NASSCOM-EY study of Indian manufacturing and energy plants found the average facility runs across 7 to 12 disconnected document systems. BIS Research linked this fragmentation to 18 to 22 percent of unplanned downtime events in Indian heavy industry, because maintenance teams make decisions without complete equipment history.

Cortex addresses this by building a unified, queryable knowledge layer on top of whatever documents a plant already has. The system ingests PDFs, spreadsheets, DOCX files, email threads, and scanned images (with OCR), extracts structured entities from them, connects those entities into a knowledge graph, and then lets operators ask natural-language questions that draw from the full document corpus, not just a keyword match against a single file.

Key numbers from the current build:

Metric	Value
Document types supported	PDF, Excel/CSV, DOCX, .eml email, image (OCR)
Chunk size	512 tokens, 50-token overlap (sliding window)
Embedding model	all-MiniLM-L6-v2, 384 dimensions, fully local
LLM tier (bulk)	gpt-oss:20b-cloud via Ollama (no install required)
LLM tier (reasoning)	gpt-oss:120b-cloud via Ollama
Knowledge graph node types	7 (Equipment, Document, Incident, Regulation, Person, Location, Procedure)
Allowed relationship types	10 (MAINTAINED_BY, LOCATED_AT, HAS_INCIDENT, and others)
Real-world test: OEM manual	Grundfos CR/CRI/CRN manual, 73 pages, 129 chunks, parsed clean
Real-world test: incident report	CSB investigation PDF, 11 pages, auto-classified as incident_report
Active modules	5 (Ingestion, Copilot, RCA, Compliance, Lessons Learned)
Infrastructure	7 Docker services, single docker-compose.yml

BACKGROUND

The Knowledge Fragmentation Problem

Industrial plants accumulate documentation over decades. A pump that has been in service for fifteen years has an OEM manual, years of maintenance work orders, inspection records, incident reports from any trips or failures, email threads where engineers discussed problems, and possibly scanned handwritten notes. None of these sources talk to each other. When a new technician needs to understand why that pump keeps vibrating, they have to locate each source individually, often across different systems, and mentally synthesize them.

This is not a retrieval problem. Search engines can find documents. The actual gap is that no single system maintains the semantic relationships between entities across documents: that pump P-101 is a KSB Etanorm 080-065-200, that it lives in Unit 3, that it was repaired in May 2024 under work order WO-2024-0512, that the root cause was misalignment introduced in February 2024 (WO-2024-0188), and that the OEM manual specifies a 3-month lubrication interval which the plant was not following. That chain of facts crosses five different documents. A basic RAG chatbot retrieves semantically similar text, not entity-linked facts.

The knowledge cliff compounds this further.

Approximately 25 percent of India's experienced industrial engineers and operators are expected to retire within the next decade. The operational judgment they carry has never been written down in a structured way. When they leave, the knowledge goes with them. A platform that continuously captures and structures knowledge from routine operational documents can act as an institutional memory system that survives personnel changes.

Architecture Overview

The system is organized into three horizontal layers: a frontend (Next.js), a backend API (FastAPI with async Python), and a storage/intelligence layer (PostgreSQL, Neo4j, Qdrant, MinIO). The LLM calls go through a local Ollama daemon, which proxies cloud-hosted models via Ollama's cloud inference API. This means no paid API keys are needed and no models need to be downloaded locally, but inference still happens at full model quality.

Service topology:

Service	Image / Tech	Host Port	Internal Port	Role
postgres	PostgreSQL 15	55432	5432	Document metadata, jobs, chat sessions
neo4j	Neo4j 5	7474/7687	7474/7687	Knowledge graph (Bolt + HTTP)
qdrant	Qdrant latest	6333/6334	6333/6334	Vector store for chunk embeddings
minio	MinIO latest	9000/9001	9000/9001	Raw file object storage
backend	Python 3.10 / FastAPI	8000	8000	API, parsers, agents, graph builder
frontend	Node 20 / Next.js 14	3000	3000	React UI, SSE client, graph viewer

PostgreSQL runs on host port 55432 (not the default 5432) because the development machine has native PostgreSQL instances on ports 5432 and 5433. Within Docker, all services address the database at the standard internal port 5432.

LLM connectivity:

The backend uses the official OpenAI Python SDK pointed at the Ollama daemon's base URL (http://localhost:11434/v1 on the host, http://host.docker.internal:11434/v1 from inside Docker). The API key is set to the literal string 'ollama'. Ollama transparently proxies requests for cloud-suffix model names (gpt-oss:20b-cloud, gpt-oss:120b-cloud) to Ollama's own inference servers. No model weights are stored locally.

One practical detail: gpt-oss wraps JSON-mode responses in markdown code fences (```json ... ```) even when json_object mode is requested. The client.py module handles this with an extract_json() function that strips the fences before parsing. Without this, every entity extraction call fails with a json.JSONDecodeError.

MODULE 1

Document Ingestion and Processing Pipeline

Ingestion is a six-stage sequential pipeline. Each stage is independent and the graph build stage is explicitly best-effort: if Neo4j is unreachable or entity extraction fails, the document still gets indexed in Qdrant and the job finishes successfully. The failure is logged but does not block the user.

Stage	Code Location	What Happens
1. Parse	backend/parsers/	File bytes dispatched by extension to PDF, Excel, DOCX, email, or image parser. OCR runs on i
2. Type Infer	parsers/base.py	If doc_type not explicit, infer from content keywords: 'work order', 'vibration', 'incident', 'OISD', etc
3. Chunk	utils/chunker.py	Sliding window over text: 512-token target, 50-token overlap. Paragraph and sentence boundarie
4. Embed	vector_store/qdrant.py	all-MiniLM-L6-v2 via sentence-transformers. 384-dim vectors. Entirely local, zero network calls. U
5. Entity Extract	graph/entity_extraction.py	gpt-oss:20b-cloud in JSON mode. Extracts equipment, persons, incidents, regulations, procedure
6. Graph Build	graph/graph_builder.py	MERGE-based upsert to Neo4j on natural keys (tag, name, code). Same entity tag across docs =

Chunker implementation detail:

The chunker tokenizes with tiktoken (cl100k_base). It tries to split at paragraph boundaries first, then sentence boundaries, then word boundaries. The _hard_split() function handles the edge case where a single paragraph exceeds the target token count (this happens with long tabular data in PDFs or densely packed procedure text). Earlier builds passed a hardcoded 512 to _split_units regardless of what the caller specified, so the overlap logic was inconsistent. That was fixed by threading max_tokens through from chunk_text() to the internal helpers.

SHA256 extraction cache:

Entity extraction is the most expensive per-document operation because it calls the LLM once per chunk (or batches of chunks). To avoid re-extracting unchanged content during re-ingestion, the system hashes the chunk text and stores the result keyed by that hash. On re-ingest of the same file, cache hits skip the LLM call entirely. This matters most during testing when the same real-world documents are ingested repeatedly.

Supported formats:

Format	Parser	Key dependencies	Notes
.pdf	pdf_parser.py	pypdf	Text-layer PDFs. Multilingual PDFs (like the 73-page Grundfos manual) parse c
.xlsx / .csv	excel_parser.py	openpyxl, pandas	Rows converted to readable text. Column headers preserved. Numeric sensor r
.docx	docx_parser.py	python-docx	Paragraph and table extraction. Heading styles used to infer structure.
.eml	email_parser.py	email stdlib	Thread-aware: extracts From, To, Date, Subject plus body of all parts. Multi-repl
image (jpg/png/tiff)	image_parser.py	pytesseract, Pillow	OCR via Tesseract. Preprocessing: grayscale + threshold. Used for scanned for

MODULE 2

Knowledge Graph Layer

The knowledge graph is the part of this system that separates it from a conventional RAG pipeline. Standard RAG retrieves text chunks by semantic similarity. If a user asks about pump P-101, it finds the chunks that mention P-101 and feeds them to the LLM. That works for answering questions about a single document. It does not work for questions like 'who else maintains the same equipment that had an incident in Unit 2' because those facts are spread across multiple documents and the relationship between them was never made explicit.

The graph makes those relationships explicit. Equipment nodes are merged on their tag (P-101, C-150, V-200). Person nodes are merged on name. Regulation nodes are merged on code (OISD-STD-105). When a second document mentions P-101, the graph builder does a MERGE, not a CREATE, so there is still exactly one P-101 node, now with two Document neighbors instead of one. After ingesting all six sample documents, P-101 has neighbors from three different source files.

Node types and natural keys:

Node Label	Natural Key (MERGE field)	Other Properties
Equipment	tag	name, description, manufacturer, location
Document	id (UUID)	title, doc_type, filename, source, created_at
Incident	id	description, date, severity, status
Regulation	code	name, body, category
Person	name	role, department
Location	name	unit, area
Procedure	title	code, revision, category

Allowed relationship types:

Relationship	From	To	Meaning
MAINTAINED_BY	Equipment	Person	Person performs maintenance on equipment
LOCATED_AT	Equipment	Location	Equipment is physically in this location
HAS_INCIDENT	Equipment	Incident	Equipment was involved in this incident
REFERENCES_REGULATION	Document	Regulation	Document cites this regulatory standard
DESCRIBES_PROCEDURE	Document	Procedure	Document describes this operating procedure
CAUSED_BY	Incident	Equipment	Incident was caused by this equipment failure
INVESTIGATED_BY	Incident	Person	Person investigated this incident
AUTHORED	Person	Document	Person authored or co-authored this document
REQUIRED_BY	Procedure	Regulation	Procedure is mandated by this regulation
SIMILAR_TO	Incident	Incident	Incidents share failure pattern (from Lessons)

Relationship allowlisting prevents the LLM from creating arbitrary relationship types. gpt-oss occasionally returns relationship types like IS_PART_OF or USES_STANDARD that are not in the list. The graph builder silently drops these rather than creating spurious edges that would clutter the visualization.

Graph queries used by agents:

- get_asset_subgraph(tag): everything within 1-2 hops of an equipment tag. Used by the Copilot and RCA agents to inject graph context alongside the Qdrant chunks.
- get_all_nodes() / get_all_edges(): full graph dump for the visualization UI. Paginated at 300 nodes and 600 edges.
- search_entities(query): case-insensitive substring search across tag, name, code, description. Powers the graph search box.
- graph_stats(): node counts by label plus total relationship count. Displayed on the dashboard.

MODULE 3

Expert Knowledge Copilot

The Copilot is the primary user-facing intelligence interface. It accepts natural-language questions and returns grounded answers with source citations, a confidence score, and detected asset tags. Responses stream token-by-token via Server-Sent Events so the user sees output appear progressively rather than waiting for the full response.

Hybrid retrieval:

The retriever runs two lookups in parallel. First, it embeds the query with all-MiniLM-L6-v2 and does a top-6 similarity search in Qdrant. Second, it scans the query text for asset tags using a regex pattern ([A-Z]{1,3}-\d{2,4}) and queries Neo4j for the 1-2 hop subgraph around each detected tag. The graph context is serialized to a structured text block and prepended to the system prompt alongside the Qdrant chunks.

This combination handles two different query types that pure semantic search handles poorly. For 'what does the OEM manual say about P-101 lubrication', Qdrant finds the relevant chunks by content similarity. For 'who else is connected to P-101 in the graph', the Neo4j query answers it directly from the relationship structure. Most real operational questions need both.

Confidence scoring:

Confidence is derived from the top Qdrant similarity score. Scores above 0.5 are HIGH, 0.3 to 0.5 are MEDIUM, and below 0.3 are LOW. This is a proxy metric, not a calibrated probability. Its main purpose is to signal to the user whether the answer is drawing from closely matched content or from distant semantic matches.

Session persistence:

Each chat session has a UUID stored in PostgreSQL. The last N messages from the session are retrieved at the start of each turn and included in the context window so the LLM has conversation history. The session ID is stored on the client side and passed with each request. A complication in the FastAPI route is that SQLAlchemy's async session cannot be shared between the HTTP request scope and the streaming generator scope. The route uses a separate get_db_context() async context manager for the generator, keeping the two scopes independent.

Streaming and React StrictMode:

The SSE protocol sends three event types in sequence: a meta event with citations and confidence, many delta events with token text, and a done event. The frontend accumulates deltas into the message content field. React 18 StrictMode double-invokes state updaters in development mode, which caused every token to be appended twice (producing 'TheThe May May' style output). The fix is immutable updates: instead of mutating content, each updater returns a new object with content set to last.content + delta. Immutable updates are idempotent under double-invocation.

MODULE 4

Maintenance RCA Agent

The RCA module takes an equipment tag and produces a structured root cause analysis drawing from the full document corpus. The output is a typed Pydantic model with immediate_cause, contributing_factors, root_causes, evidence, recommendations, and a confidence field.

For pump P-101, the expected RCA output (based on the six sample documents) is: immediate cause is outer-race spalling on the SKF 6310 drive-end bearing. Root cause 1 is shaft misalignment introduced during the February 2024 coupling replacement. Root cause 2 is lubrication interval at 6 months versus the OEM-specified 3 months. Evidence comes from three sources: the maintenance work order (vibration readings), the OEM manual (lubrication specification), and the inspection CSV (vibration trend showing two ALARM readings before repair). The agent synthesizes all three without any manual cross-referencing.

For compressor C-150, the RCA identifies the cracked 2nd-stage discharge valve plate as immediate cause, valve plates at 9,200 hours against an 8,000-hour OEM replacement interval as root cause 1, and the absence of automated alarm-to-work-order escalation as root cause 2. The evidence chain specifically includes the rod-drop reading (0.48 mm on July 26, 2 days before the trip) which was logged but not escalated.

MODULE 5

Compliance Intelligence

The compliance module scans all ingested documents against a fixed checklist of six requirements drawn from the Indian industrial regulatory frameworks relevant to the problem statement: OISD-STD-105, OISD-STD-106, PESO, Factory Act 1948, and OISD-STD-118.

#	Framework	Requirement Summary	Expected Statu
1	OISD-STD-105	Preventive maintenance program for rotating equipment with defined intervals	PARTIAL
2	OISD-STD-106	Pressure vessel and relief valve inspection and certification records	PARTIAL
3	PESO	Hazardous area classification and licensing for flammable storage	MET
4	Factory Act 1948	Permit-to-work / lockout-tagout system for maintenance activities	MET
5	Factory Act 1948	Periodic safety inspections and worker PPE compliance records	PARTIAL
6	OISD-STD-118	Incident reporting, investigation, and root-cause closure process	MET

Items 1 and 2 are PARTIAL because the evidence shows the processes exist but had known gaps at the time of the most recent documents: P-101's lubrication interval was wrong (6 months instead of 3), and V-200's relief valve re-certification was pending as of the email thread. Item 5 is PARTIAL because PPE requirements appear in the SOP but no dedicated audit trail document was ingested.

Each checklist item is evaluated independently by calling the LLM with all relevant document chunks as context. The LLM returns a JSON object with status, risk_level, evidence_found, gaps, and recommendations. The overall compliance score is a weighted average of the individual statuses.

MODULE 6

Lessons Learned Engine

The Lessons Learned module serves two functions: pattern discovery across all ingested incidents, and similarity search to find incidents that resemble a described failure mode.

Pattern discovery scans incident-type chunks and asks the LLM to identify recurring themes across the corpus. With the six sample documents, it finds three patterns: lubrication interval deviation causing bearing failure (P-101), overdue component replacement causing equipment trip (C-150 valve plates), and inspection finding not escalated to work order (C-150 rod drop). As more documents are ingested, the engine surfaces cross-site patterns that no individual document would reveal.

Similarity search uses Qdrant directly: embed the query text, search against incident-type chunks, return the top matches with scores. A search for 'high temperature compressor trip' should return INC-2024-0731 (C-150, 154 degrees C trip) with a score above 0.75.

FRONTEND

Interface Design and Implementation

The frontend is built with Next.js 14 (App Router), React 18, TypeScript, and Tailwind CSS. The design system is called Foundry: a warm near-black base color, a single amber/ember accent (#e8833a), engineering dot-grid backdrop, JetBrains Mono for data values and labels, Inter for body text, and solid panels with hairline borders. The design explicitly avoids the purple-to-cyan gradient look that is common in 2024-2026 AI product templates.

Pages and routes:

Route	Component	Function
/	page.tsx	Dashboard: graph stats, recent documents, quick-start suggestions
/upload	upload/page.tsx	Document upload with animated Pipeline component (Parse > Chunk > Embed > Graph stages)
/chat	chat/page.tsx	Expert Copilot: SSE chat, confidence badge, citations, asset tag chips
/graph	graph/page.tsx	Force-directed graph visualization of Neo4j entities and relationships
/documents	documents/page.tsx	Ingested document list with type badges and chunk counts
/compliance	compliance/page.tsx	Compliance scan results with MET/PARTIAL/GAP color coding
/rca/[tag]	rca/page.tsx	RCA result for a specific asset tag: immediate cause, root causes, evidence
/lessons	lessons/page.tsx	Failure pattern list and similarity search interface

Markdown rendering:

Copilot answers are rendered with react-markdown and remark-gfm. A clean() function runs before rendering to normalize citation tokens: the LLM sometimes outputs Jupyter-notebook-style citations like [1 dagger L7-L9] enclosed in Japanese corner brackets, which are stripped and replaced with clean [1] references.

Language Model Layer

All LLM calls go through backend/llm/client.py. The module provides three public functions: complete() for non-streaming text, complete_json() for JSON-mode responses, and stream() for token-by-token streaming. The model tier (FAST or SMART) is a parameter on each call.

Function	Tier	Temperature	Model	Used for
complete_json()	FAST	0.1	gpt-oss:20b-cloud	Entity extraction, doc type inference, compliance evaluation, RCA
stream()	SMART	0.3	gpt-oss:120b-cloud	Copilot answers, RCA narrative, lessons pattern text
complete()	FAST	0.2	gpt-oss:20b-cloud	Short factual lookups, confidence scoring

The FAST/SMART split is a rate-limit and cost management strategy. Entity extraction runs once per document chunk during ingestion and is cached, so it can use the heavier SMART model. But compliance scans evaluate six checklist items per scan, and RCA pre-processing does multiple structured lookups, so those use the faster 20b model to stay within free-tier usage limits during a live demo.

JSON fence stripping:

- gpt-oss:20b-cloud and gpt-oss:120b-cloud wrap JSON-mode responses in markdown code fences.
- The raw response looks like: ```json\n{...}\n``` rather than just the bare JSON object.
- extract_json() in client.py strips the opening ```json and closing ``` before calling json.loads().
- If stripping fails, a fallback span-detection routine finds the first { and last } and parses between them.
- Without this, 100% of entity extraction calls fail silently with a JSONDecodeError.

DATA MODEL

PostgreSQL Schema

PostgreSQL holds three main tables: documents, ingestion_jobs, and chat_sessions (with a chat_messages child table). The database is accessed via SQLAlchemy with asyncpg as the async driver. All schemas are defined as SQLAlchemy ORM models in backend/db/models.py and created with Alembic migrations.

Table	Key Columns	Purpose
documents	id (uuid pk), title, doc_type, filename, file_path, chunk_count, status, created_at	One row per ingested file. Status tracks processing state.
ingestion_jobs	id (uuid pk), document_id (fk), status, error_message, started_at, finished_at	One row per background ingestion job. Polled by the upload UI.
chat_sessions	id (uuid pk), created_at, updated_at	Groups messages into a conversation thread.
chat_messages	id (uuid pk), session_id (fk), role, content, citations, confidence, created_at	Individual messages with citations stored as JSONB.

VALIDATION

Real-World Document Testing

The system was tested against two categories of documents: a curated set of six synthetic industrial documents (with cross-linked entities and known ground truth), and two real-world PDFs downloaded from public sources.

Synthetic test corpus (6 files):

File	Type	Key Entities
pump_P-101_maintenance_report.txt	maintenance_report	P-101, SKF 6310, WO-2024-0512, R. Sharma, A. Mehta
OEM_manual_P-101_excerpt.txt	oem_manual	P-101, KSB Etanorm 080-065-200, ISO 10816-3
incident_report_C-150_compressor.txt	incident_report	C-150, INC-2024-0731, Ariel JGK/4, OISD-STD-118
safety_procedure_hot_work.txt	safety_procedure	SOP-SAF-014, PESO, Factory Act 1948, P-101, V-200, C-150
email_thread_V-200_relief_valve.eml	email	V-200, OISD-STD-106, S. Nair, A. Mehta
inspection_records.csv	inspection	P-101, V-200, C-150, E-301 (sensor readings over time)

Real-world PDFs tested:

File	Source	Size	Pages	Chunks	Classified as
grundfos_CR_pump_manual.pdf	Grundfos	9.8 MB	73	129	oem_manual (auto)
csb_investigation_report.pdf	U.S. CSB	1.4 MB	11	5	incident_report (auto)

The Grundfos manual is a deliberately hard test case. It is 73 pages, multilingual (29 language sections), and the valuable technical content is concentrated in the English section (pages 4-26). The parser handles it correctly because pypdf extracts text by character stream rather than by page layout, so the multilingual table of contents and distributor address appendix produce irrelevant chunks but do not corrupt the English technical content. The 129 chunks include the bearing replacement intervals, shaft seal specifications, fault-finding table, and operating limits, all of which are verifiable against the Q&A; ground truth in QNA_GRUNDFOS_MANUAL.md.

Cross-document linking verification:

After ingesting all six synthetic documents, the Neo4j graph contains one P-101 Equipment node with Document neighbors from three different source files (maintenance report, OEM manual, and hot work SOP). This confirms that MERGE-on-tag is working: the same physical asset is represented by a single node regardless of how many documents mention it.

DEPLOYMENT

Infrastructure and Startup

The full stack runs in Docker Compose. A single docker-compose.yml defines all seven services with their environment variables, port mappings, volume mounts, and health checks. The backend's OLLAMA_BASE_URL is set to http://host.docker.internal:11434/v1 with an extra host entry (host-gateway) so it can reach the Ollama daemon running on the host machine. The frontend communicates with the backend via the NEXT_PUBLIC_API_URL environment variable.

Standard startup sequence:

- Step 1: docker compose up -d (starts all 7 services; first run pulls images and builds containers, approximately 3-5 minutes)
- Step 2: docker compose logs -f backend (watch for 'Application startup complete' before using the UI)
- Step 3: Open http://localhost:3000 for the frontend, http://localhost:8000/docs for the FastAPI Swagger UI
- Step 4: Neo4j Browser at http://localhost:7474 (credentials: neo4j / change_me_neo4j)
- Step 5: MinIO console at http://localhost:9001 (credentials: minioadmin / change_me_minio)
- For development iteration on the frontend, run 'npm run dev' instead of npm start to avoid .next cache corruption when rebuilding while the server is running

Environment variables (key values):

Variable	Value	Notes
OLLAMA_API_KEY	ollama	Literal string; Ollama does not validate keys
OLLAMA_BASE_URL	http://localhost:11434/v1	Host-side; use host.docker.internal from Docker
LLM_MODEL_FAST	gpt-oss:20b-cloud	Bulk operations and structured JSON extraction
LLM_MODEL_SMART	gpt-oss:120b-cloud	Reasoning and streaming answers
POSTGRES_URL	postgresql+asyncpg://...@localhost:55432/..	Port 55432 to avoid host PostgreSQL conflict
NEO4J_URI	bolt://localhost:7687	Bolt protocol for graph queries
QDRANT_URL	http://localhost:6333	REST API for vector operations

ENGINEERING LOG

Issues Encountered and Resolutions

This section documents the non-trivial technical problems encountered during development and how they were resolved. It is included because the solutions represent real engineering decisions rather than textbook implementations.

Issue	Symptom	Root Cause	Fix
gpt-oss JSON fence wrapping	json.JSONDecodeError on every extraction	Model wraps output in ```json fences	extract_json() strips fences before parsing; fallback span detection
PostgreSQL port conflict	Connection refused on 5432	Host has native PG v13 and v17 on 5432/5433	Container mapped to host port 55432
config.py .env path	Settings not loading in backend	Path resolved relative to CWD (backend/) not project root	Path(__file__).resolve().parent.parent / '.env'
Chunker oversized paragraphs	Chunks exceeding 512 tokens split output	_split_units used hardcoded 512 instead of max_tokens	Added max_tokens parameter and passed it through call chain
.txt no type inference	All .txt classified as 'general'	Early return for txt skipped infer_doc_type()	Moved type inference before the early return
Blank graph labels	Document and Incident nodes showed no labels	UI displayed empty label/name fields	display_key() added title and description fallbacks
React StrictMode double-invoke	Chat shows 'TheThe May May'	Mutable message object updated last.content += token twice	Immutable update: {...last, content: last.content + token}
Frontend .next corruption	MODULE_NOT_FOUND after rebuild	Rebuild while next start was still running	Always stop server, rm -rf .next, rebuild, restart

EVALUATION

Judging Criteria Alignment

The problem statement specifies five judging criteria with defined weights. This section maps Cortex's implementation against each criterion.

Criterion	Weight	How Cortex Addresses It
Innovation	25%	Hybrid RAG + graph retrieval is not common in hackathon submissions. Most teams build pure RAG chatbots. Th
Business Impact	25%	Directly addresses the 35% time-on-search problem and the knowledge cliff. The compliance module maps to rea
Technical Excellence	20%	6-stage pipeline, 7 node types, 10 relationship types, 5 parser formats, 2 LLM tiers, SHA256 extraction cache, im
Scalability	15%	Stateless backend (jobs are async), vector DB scales horizontally, Neo4j handles enterprise graph sizes. The do
User Experience	15%	Foundry design language (not a template). Animated pipeline stages on upload. Streaming answers with citations

The system runs entirely on free infrastructure: Ollama cloud models require no paid API keys, all database software is open-source, and the Docker Compose setup runs on a standard developer laptop. This means the demo can run without any network dependencies beyond the Ollama cloud inference endpoint.

PROJECT STRUCTURE

Repository Layout

```
project root/
  backend/
    agents/          copilot.py  rca.py  compliance.py  lessons.py  retriever.py
    db/              models.py  database.py  migrations/
    graph/           neo4j_client.py  entity_extraction.py  graph_builder.py  queries.py
    llm/             client.py  (OpenAI SDK -> Ollama proxy)
    parsers/         pdf_parser.py  excel_parser.py  docx_parser.py  email_parser.py  image_parser.
py
    routers/         chat.py  ingest.py  graph.py  documents.py  compliance.py  lessons.py
    services/        ingestion.py
    utils/           chunker.py
    vector_store/    qdrant.py
    config.py        (Pydantic-settings, .env from project root)
    main.py          (FastAPI app, CORS, router registration)
  frontend/
    src/app/         page.tsx  chat/  upload/  graph/  documents/  compliance/  rca/  lessons/
    src/components/  Markdown.tsx  Pipeline.tsx  Backdrop.tsx  ui/Badge.tsx
    src/lib/         api.ts  (streamChat, fetchGraph, runRCA, ...)
    tailwind.config.ts
    globals.css      (.panel, .eyebrow, .backdrop-grid, prose-chat)
  sample_data/       6 synthetic files + real_world/ subfolder
  eraser_diagrams/   5 .eraserdiagram files (paste at app.eraser.io)
  docker-compose.yml
  .env.example
  QNA_GROUND_TRUTH.md
  QNA_GRUNDFOS_MANUAL.md
  START.md
  DEMO_SCRIPT.md
  ARCHITECTURE.md
```

APPENDIX

Eraser.io Diagram Files

Five architecture diagrams are provided as Eraser.io DSL files in the eraser_diagrams/ folder. To render them, open app.eraser.io, create a new diagram of the matching type, and paste the file contents into the code editor.

File	Diagram Type	Contents
01_system_architecture.eraserdiagram	Cloud Architecture	Full service topology: Frontend, Backend, LLM, Storage layers with connections
02_ingestion_pipeline.eraserdiagram	Flowchart	6-stage ingestion: Upload > Parse > TypeInfer > Chunk > Embed > Entity Extract > G
03_knowledge_graph_schema.eraserdiagram	Entity Relationship	7 node types with properties and all 10 named relationship types
04_agent_architecture.eraserdiagram	Flowchart	5 agents with routing, retrieval split (Qdrant + Neo4j), and LLM tier dispatch
05_rag_retrieval_sequence.eraserdiagram	Sequence Diagram	Full SSE chat flow: User > Frontend > FastAPI > Retriever > Qdrant + Neo4j > LLM >

Document generated: July 07, 2026 at 18:21. All technical specifications reflect the codebase state at the time of report generation.